

Tool Experience Report: AutoGen and MetaGPT

Oliver Browne

Papers: Wu et al., *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*; Hong et al., *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework* (ICLR '24)

Tools used: AutoGen AgentChat 0.7.5 and MetaGPT 0.8.2

Date of experiments: 30 September 2026

How I ran this. I used an AI coding assistant to drive the terminal on my Windows machine, and kept every command, prompt, raw model response, generated file and test log. No model responses were mocked, and no generated code was edited by hand before testing.

1 Your Experience Using the Tool

Setup

I had no paid API key, so instead of GPT-4 I ran **Qwen2.5-Coder-1.5B-Instruct** (4-bit) on **llama.cpp** as a local OpenAI-compatible endpoint, CPU-only on a Ryzen 7 5800X with 32 GB of RAM and an 8K context. This is not a reproduction of the papers' numbers; it tests how much of each framework's value comes from the framework and how much from the model. Both tools got the same task, `merge_intervals`, with four traps: don't mutate the input, don't alias its inner lists, don't let a contained interval shrink its enclosing one, and raise `ValueError` on reversed intervals. I wrote the unit tests and recorded their hashes before any model call, so the model could not shape them.

AutoGen installed easily (28 packages in about 6 seconds), but version 0.7.x is a rewritten async API, so the paper's `ConversableAgent/UserProxyAgent` code no longer runs. Pointing it at the local model only needed a `base_url` and a placeholder API key. `LocalCommandLineCodeExecutor` failed with `WinError 5` inside my Windows sandbox and only worked outside it.

MetaGPT took eight install attempts, six of which failed. It requires Python below 3.12, and its `lancedb==0.4.0` pin no longer exists on PyPI, so the resolver silently fell back to MetaGPT 0.1, which then failed to build. `volcengine-python-sdk` also failed to build because of Windows path-length limits. The working install needed Python 3.11, two manual version overrides and 218 packages, and the dependency checker still flags both overrides. Configuration was simple once I redirected `CONFIG_ROOT` to a local folder instead of my home directory.

AutoGen

I built a `RoundRobinGroupChat` with an LLM coder, an LLM reviewer told not to write code, and a non-LLM verifier that ran my 11 tests and posted the results, capped at 9 turns at temperature 0.

Over 76.8s and 6 LLM calls, the coder fixed the nested-interval bug but introduced a new aliasing bug, never stopped sorting in place or added validation, and stayed at 8/11 on every attempt. After the verifier posted failures, the coder blamed a test that was actually passing and then resubmitted identical code. The reviewer approved all three versions, claiming the code didn't mutate the input and raised `ValueError`, both false. The framework itself worked as described, and the only participant that told the truth was the one with no LLM.

MetaGPT

Following the official *MultiAgent 101* tutorial, I built a two-role **Team**: an **Implementer** watching **UserRequirement** and a **Reviewer** watching the Implementer's output. This exercises MetaGPT's role/action publish-subscribe routing but not the SOP documents (PRD, design, API spec) that are the paper's main contribution. The team ran in 4.65s with 2 LLM calls. The generated code sorted the input in place and tried to assign into tuples, passing only 3 of 10 tests. The Reviewer's entire output was **PASS**.

2 Strengths

AutoGen's best feature is mixing LLM and non-LLM participants in one conversation, which let me put a deterministic verifier inside the loop, and in my run that was the only part that produced correct information. It was also easy to point at a local model, quick to install, and highly observable, with per-message sources, per-agent token counts and explicit stop reasons. MetaGPT's role/action/watch model expresses a workflow declaratively, routing by message type rather than hard-coded turn order, and it worked right away once installed. Its fixed-stage structure needs only one call per role, so it ran in under 5s against AutoGen's 77s. It also has built-in team budgets and cost tracking, and ships many bundled tools well beyond the paper's scope.

3 Weaknesses

In both tools an LLM reviewer was not verification: each confidently approved broken code, and neither framework's defaults require running tests in the loop. AutoGen's API has changed so much that the paper is now more of a concept document, and its repair loop has no way to notice an agent repeating the same wrong code. It also makes it easy to confuse "the conversation finished" with "the task succeeded", as my own harness did by exiting 0 on failure. MetaGPT's packaging is fragile (exact pins, a Python ceiling and heavy cloud SDKs in the core install), and the silent fallback to version 0.1 could leave an unwary user with a completely different tool. Custom teams run no code by default, and the SOP pipeline assumes large-context models, with no lite mode for small local ones.

4 Potential for Practical Impact

Both tools turn LLM workflows into versionable, testable software artifacts. AutoGen's lasting contribution is treating humans, tools and LLMs as equal conversation participants, which suits

researchers prototyping multi-agent patterns and teams that need human approval steps. MetaGPT’s is that structured intermediate artifacts like PRDs reduce hallucination cascades, which suits developers wanting scaffolded greenfield prototypes and educators teaching what design documents look like. The main barriers are trust (a reviewer that approves broken code creates false confidence), cost (about 9K prompt tokens for a 10-line function), and maintainability given AutoGen’s rewrite and MetaGPT’s dependency rot. Running generated code safely and depending on GPT-4-class models are further hurdles for enterprises that must run locally.

5 Reflection and Recommendations

Using the tools showed that orchestration is the easy part: both frameworks routed messages and ended cleanly, and every quality failure came from the agents. In AutoGen, adding a reviewer made things worse by adding a confident, wrong voice rather than a correction. Running both on the same task made the trade-off clear: AutoGen’s loop could in principle recover but was slow, while MetaGPT’s pipeline was fast but had no recovery path at all. For AutoGen, I recommend a built-in stagnation guard that detects repeated code after a failure and then stops, escalates to a human, or injects the failing behaviour. For MetaGPT, I recommend making executable verification a default step of every code-producing Team, and fixing the packaging by moving cloud SDKs into optional extras and replacing exact pins with ranges.

Evidence summary

	AutoGen 0.7.5	MetaGPT 0.8.2
Install attempts / failures	2 / 1 (network only)	8 / 6
Packages installed	28	218 (+2 overrides)
Python	3.13	3.11 (requires < 3.12)
Roles	coder, reviewer, non-LLM verifier	Implementer, Reviewer
LLM calls / wall time	6 / 76.8 s	2 / 4.65 s (team)
Final tests passing	8 / 11 (unchanged from baseline)	3 / 10
Reviewer verdict	“correct” ×3 (wrong)	PASS (wrong)
Task outcome	Fail	Fail